

7-2 · 텍스트 데이터 Chunking · 실습 1.5H

Document Loader와 텍스트 분할 실습

문서를 읽고 자르고, 잘린 경계를 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① TextLoader 로 한글 문서를 utf-8 로 깨짐 없이 로드한다
- ② 틀린 인코딩(cp949)으로 읽어 오류 메시지를 미리 본다
- ③ chunk_size 200 · overlap 30 으로 잘라 경계와 겹침을 확인한다
- ④ 메타데이터가 조각마다 따라오는 것을 확인하고 직접 추가한다

7단계에서 Character 분할기와 비교해 Recursive 가 기본인 이유를 봅니다

일곱 단계 — ~/data-eng/week7 한 폴더에서

7-1 의 가상환경을 그대로 켜고 시작합니다 — Windows(WSL2)·macOS·리눅스 모두 같습니다

1. 준비	2. 로드	3. 실패	4.5. 분할·검침	6.7. 메타·비교
pip install notes.txt	TextLoader utf-8	cp949 로 일부러	200 / 30 이웃 대조	metadata Character

패키지 설치와 실습 문서 만들기

1

가상환경을 켜다

`cd ~/data-eng/week7 → source .venv/bin/activate`

2

패키지 세 개 설치

`pip install langchain-community langchain-text-splitters pypdf`

3

notes.txt 를 만든다

`cat > notes.txt <<'EOF' — 1장·2장 두 문단짜리 한글 문서`

4

cat notes.txt 로 확인

한글이 깨지지 않고 출력되면 준비 끝

실습 문서

notes.txt

1장·2장

핵심 명령 — 로드 · 분할 · 확인

로드	<code>TextLoader("notes.txt", encoding="utf-8")</code>
실행·확인	<code>.load() → len(docs) · docs[0].metadata</code>
분할기	<code>chunk_size=200, chunk_overlap=30</code>
분할	<code>chunks = splitter.split_documents(docs)</code>
조각 확인	<code>len(chunks) · len(c.page_content)</code>

PDF 는 `PyPDFLoader("sample.pdf").load()` — 페이지당 Document 1개로 로드된다

encoding="cp949" 로 읽으면 — 오류를 미리 본다

3-1 의 CSV 인코딩 문제가 여기서도 나옵니다 — Windows 에서 만든 한글 파일은 cp949 인 경우가 많습니다

try 로 감싸	except 에서	보이는 것	교훈
<pre>TextLoader encoding="cp949"</pre>	<pre>type(e).__name__ str(e)[:200]</pre>	<pre>UnicodeDecodeError 또는 글자 깨짐</pre>	encoding 을 늘 명시하는 습관

잘린 경계와 겹침 — 눈으로 확인

4단계 — 분할 결과

각 조각 200자 이하, 경계가 문단·문장 끝인지

- `print("조각 수:", len(chunks))`
- chunk i (N자) 머리글과 본문을 차례로
- 잘린 곳이 문단·문장 끝이면 성공

VS

5단계 — overlap 대조

이웃 조각의 끝 40자와 시작 40자를 나란히

- `chunks[i].page_content[-40:]`
- `chunks[i + 1].page_content[:40]`
- 같은 문장이 겹치면 동작 · 안 겹치면 자연 경계

메타데이터가 따라오는지, 직접 더하기

- 1 print(i, c.metadata) 로 전부 확인**
모든 조각에 source 가 붙어 있다 — PDF 라면 page 도 함께
- 2 c.metadata["chunk_id"] = i**
조각 번호를 직접 추가
- 3 c.metadata["chars"] = len(c.page_content)**
글자 수도 함께 — 7-3 벡터DB 에 저장할 때 같이 들어간다

출처의 쓸모

어느 문서
몇 쪽

두 분할기 비교 — Recursive 가 기본인 이유

RecursiveCharacterTextSplitter

구분자 여러 개를 차례로 시도 — 기본 선택



VS

CharacterTextSplitter

separator 하나만 — 긴 문단은 못 잘라 큰 조각이 남는다

- separator="\n\n" 하나
- chunk_size=200, chunk_overlap=30 은 같게
- 문단이 200자를 넘어도 그대로 남는다

이 1.5시간이 끝날 때 확인할 것

- 1 `cat notes.txt` 에 한글이 깨지지 않고 나온다
- 2 `docs[0].metadata` 에 `{'source': 'notes.txt'}` 가 있다
- 3 `cp949` 로 읽으면 `UnicodeDecodeError` 가 난다
- 4 각 조각이 200자 이하이고 경계가 대체로 문단·문장 끝이다
- 5 이웃 조각의 끝과 시작에 같은 문장이 겹쳐 보인다
- 6 `Character` 분할기는 200자보다 큰 조각이 남을 수 있다

7-2 · 실습 1.5H 마무리

다음은 Chunk 파라미터 튜닝 (1H)

긴 문서로 size·overlap 조합을 비교해 재고, 우리 문서에 맞는 기준값을 근거와 함께 정합니다

✓ long_notes.txt 수천 자 문서 만들기

✓ 다섯 조합의 조각 수·평균·최대 길이

✓ 문장 중간 절단 세기 · 키워드 검색 비교

✓ 기준값 표에 선택과 근거 기록

가져갈 것

notes.txt 와 chunks 를 만든 코드